DCIA PoC — Developer Instructions with Sample Data

Design Change Intelligence Assistant — Proof of Concept

Source: MDL Problem Statements PS-1 and PS-2 · Architecture Reference v1.0

PoC objectives

By end of PoC, demonstrate:

1. A user types a natural language design change request and receives a structured impact report
2. Neo4j graph traversal correctly identifies 2–3 hop cross-discipline cascades
3. A parametric cost estimate with confidence band is returned
4. At least one Class rule compliance flag is surfaced with citation
5. The React UI renders an impact heat-map and cost breakdown

Timebox: 4–6 weeks · **Team:** 1 backend engineer, 1 frontend engineer, 1 data/graph engineer

---

Prerequisites — local dev environment

```bash
# Required tools
node >= 18.x
python >= 3.11
docker + docker-compose
neo4j desktop (free) or neo4j aura free tier
openai api key OR anthropic api key

# Clone the PoC repo skeleton (create this structure)
mkdir dcia-poc && cd dcia-poc
mkdir backend frontend graph-seed data
```

Step 1 — Spin up Neo4j and seed the knowledge graph

1a. Start Neo4j via Docker

```yaml
# docker-compose.yml
version: "3.8"
services:
  neo4j:
    image: neo4j:5.15
    ports:
      - "7474:7474"   # Browser UI
      - "7687:7687"   # Bolt protocol
    environment:
      NEO4J_AUTH: neo4j/dcia_poc_pass
      NEO4J_PLUGINS: '["apoc"]'
    volumes:
      - ./neo4j_data:/data
```

```bash
docker-compose up -d
# Access browser at http://localhost:7474
# Login: neo4j / dcia_poc_pass
```

1b. Seed sample ship system nodes

Run the following Cypher in the Neo4j browser (or via a seed script):

```cypher
```

```
// ── Ship systems ──────────────────────────────────────────
CREATE (prop:ShipSystem  {id:"SYS-001", name:"Main Propulsion", discipline:"propulsi
CREATE (struct:ShipSystem {id:"SYS-002", name:"Hull Structure",   discipline:"struct
CREATE (elec:ShipSystem  {id:"SYS-003", name:"Electrical Power",  discipline:"electr
CREATE (wpn:ShipSystem   {id:"SYS-004", name:"Weapons Systems",   discipline:"weapon
CREATE (out:ShipSystem   {id:"SYS-005", name:"Outfitting / HVAC", discipline:"outfit
CREATE (shaft:ShipSystem {id:"SYS-006", name:"Shafting & Gearbox",discipline:"propul
CREATE (fuel:ShipSystem  {id:"SYS-007", name:"Fuel System",       discipline:"outfit

// ── Components ────────────────────────────────────────────
CREATE (eng:Component    {id:"CMP-101", name:"Main Engine MTU 4000",  partNumber:"MT
CREATE (gen:Component    {id:"CMP-102", name:"Shaft Generator 2MW",   partNumber:"AB
CREATE (mnt:Component    {id:"CMP-103", name:"Engine Mounting Frame",  partNumber:"M
CREATE (cool:Component   {id:"CMP-104", name:"Seawater Cooling Pump",  partNumber:"H
CREATE (fuel_pump:Component {id:"CMP-105", name:"Fuel Booster Pump",   partNumber:"H

// ── BOM items ─────────────────────────────────────────────
CREATE (bom1:BOMItem {id:"BOM-001", revision:"C", quantity:1, unit:"ea", model_ref:"
CREATE (bom2:BOMItem {id:"BOM-002", revision:"B", quantity:1, unit:"ea", model_ref:"

// ── Class rules ───────────────────────────────────────────
CREATE (lr1:ClassRule  {id:"CR-LR-001", society:"LR",  clause:"Pt3 Ch2 Sec5.3",  des
CREATE (lr2:ClassRule  {id:"CR-LR-002", society:"LR",  clause:"Pt3 Ch4 Sec2.1",  des
CREATE (dnv1:ClassRule {id:"CR-DNV-001",society:"DNV", clause:"RU-SHIP Pt4 Ch2", des
CREATE (dnv2:ClassRule {id:"CR-DNV-002",society:"DNV", clause:"RU-SHIP Pt4 Ch6", des

// ── Suppliers ─────────────────────────────────────────────
CREATE (mtu:Supplier  {id:"SUP-001", name:"MTU Friedrichshafen", country:"DE", leadT
CREATE (abb:Supplier  {id:"SUP-002", name:"ABB Marine",          country:"SE", leadT

// ── Historical change records ─────────────────────────────
CREATE (hc1:DesignChange {id:"HC-2022-014", description:"Engine upgrade from MTU 360
        status:"closed", date:"2022-08-15", cost_actual_usd:3200000, schedule_delay_
        summary:"Power increase 12%. Required shaft re-alignment, electrical load re
CREATE (hc2:DesignChange {id:"HC-2021-007", description:"Generator set replacement 1
        status:"closed", date:"2021-03-10", cost_actual_usd:980000, schedule_delay_d
        summary:"Switchboard upgrade required. No hull impact. Class survey triggere
CREATE (hc3:DesignChange {id:"HC-2023-031", description:"Fuel system capacity increa
        status:"closed", date:"2023-11-20", cost_actual_usd:450000, schedule_delay_d
        summary:"Tank structural reinforcement. Outfitting reroute. No propulsion im

// ── Cost records ──────────────────────────────────────────
CREATE (cr1:CostRecord {id:"CST-001", category:"labour",   amount_usd:420000,  year:
```

```
CREATE (cr2:CostRecord {id:"CST-002", category:"material", amount_usd:2400000, year:
CREATE (cr3:CostRecord {id:"CST-003", category:"testing",  amount_usd:380000,  year:
```

1c. Seed relationships

```cypher
```

```
// Component → System memberships
MATCH (eng:Component {id:"CMP-101"}), (prop:ShipSystem {id:"SYS-001"}) CREATE (eng)-
MATCH (gen:Component {id:"CMP-102"}), (prop:ShipSystem {id:"SYS-001"}) CREATE (gen)-
MATCH (mnt:Component {id:"CMP-103"}), (struct:ShipSystem {id:"SYS-002"}) CREATE (mnt
MATCH (cool:Component {id:"CMP-104"}), (out:ShipSystem {id:"SYS-005"}) CREATE (cool)
MATCH (fuel_pump:Component {id:"CMP-105"}), (fuel:ShipSystem {id:"SYS-007"}) CREATE

// BOM linkages
MATCH (eng:Component {id:"CMP-101"}), (bom1:BOMItem {id:"BOM-001"}) CREATE (eng)-[:R
MATCH (gen:Component {id:"CMP-102"}), (bom2:BOMItem {id:"BOM-002"}) CREATE (gen)-[:R

// ── CASCADE relationships (the core propagation graph) ──────
// Propulsion → Structure (weight, vibration, frame loads)
MATCH (prop:ShipSystem {id:"SYS-001"}), (struct:ShipSystem {id:"SYS-002"})
CREATE (prop)-[:CASCADES_TO {reason:"Weight & vibration transfer to hull frames", se

// Propulsion → Shafting
MATCH (prop:ShipSystem {id:"SYS-001"}), (shaft:ShipSystem {id:"SYS-006"})
CREATE (prop)-[:CASCADES_TO {reason:"Torque and alignment dependency", severity:"HIG

// Propulsion → Electrical (shaft generator output change)
MATCH (prop:ShipSystem {id:"SYS-001"}), (elec:ShipSystem {id:"SYS-003"})
CREATE (prop)-[:CASCADES_TO {reason:"Shaft generator output affects electrical load

// Propulsion → Fuel (higher power = higher fuel consumption rate)
MATCH (prop:ShipSystem {id:"SYS-001"}), (fuel:ShipSystem {id:"SYS-007"})
CREATE (prop)-[:CASCADES_TO {reason:"Fuel flow rate and tank sizing affected", sever

// Structure → Outfitting (pipe reroutes around new stiffeners)
MATCH (struct:ShipSystem {id:"SYS-002"}), (out:ShipSystem {id:"SYS-005"})
CREATE (struct)-[:CASCADES_TO {reason:"Hull stiffening may require pipe/HVAC rerouti

// Electrical → Weapons (power bus dependency)
MATCH (elec:ShipSystem {id:"SYS-003"}), (wpn:ShipSystem {id:"SYS-004"})
CREATE (elec)-[:CASCADES_TO {reason:"Weapons systems draw from main power bus", seve

// ── Class rule governance ──────────────────────────────────
MATCH (prop:ShipSystem {id:"SYS-001"}), (lr1:ClassRule {id:"CR-LR-001"}) CREATE (pro
MATCH (shaft:ShipSystem {id:"SYS-006"}), (lr2:ClassRule {id:"CR-LR-002"}) CREATE (sh
MATCH (struct:ShipSystem {id:"SYS-002"}), (dnv1:ClassRule {id:"CR-DNV-001"}) CREATE
MATCH (elec:ShipSystem {id:"SYS-003"}), (dnv2:ClassRule {id:"CR-DNV-002"}) CREATE (e

// ── Supplier linkages ──────────────────────────────────────
```

```
MATCH (eng:Component {id:"CMP-101"}), (mtu:Supplier {id:"SUP-001"}) CREATE (eng)-[:S
MATCH (gen:Component {id:"CMP-102"}), (abb:Supplier {id:"SUP-002"}) CREATE (gen)-[:S

// ── Historical change → system impacts ──────────────────────
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (prop:ShipSystem {id:"SYS-001"}) CREATE
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (struct:ShipSystem {id:"SYS-002"}) CREA
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (shaft:ShipSystem {id:"SYS-006"}) CREAT
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (cr1:CostRecord {id:"CST-001"}) CREATE
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (cr2:CostRecord {id:"CST-002"}) CREATE
MATCH (hc1:DesignChange {id:"HC-2022-014"}), (cr3:CostRecord {id:"CST-003"}) CREATE
```

1d. Verify the graph

```cypher
// Should return 5 systems with cascade paths
MATCH (a:ShipSystem)-[r:CASCADES_TO]->(b:ShipSystem)
RETURN a.name, r.severity, b.name

// Test the core PoC query — propulsion change, 3-hop traversal
MATCH path = (s:ShipSystem {id:"SYS-001"})-[:CASCADES_TO*1..3]->(affected:ShipSystem
RETURN affected.name, affected.discipline, length(path) AS hops,
       [r IN relationships(path) | r.reason] AS cascade_reasons
ORDER BY hops
```

Step 2 — Python backend (FastAPI)

2a. Project structure

```
backend/
├── main.py
├── routers/
│   ├── chat.py          # NL query endpoint
│   ├── impact.py        # Graph traversal endpoint
│   └── cost.py          # Cost estimation endpoint
├── services/
│   ├── neo4j_service.py
│   ├── llm_service.py
│   └── cost_model.py
├── models/
```

```
|   └── schemas.py
└── requirements.txt
```

2b. requirements.txt

```
fastapi==0.111.0
uvicorn==0.29.0
neo4j==5.19.0
openai==1.30.0        # or anthropic==0.25.0
python-dotenv==1.0.1
pydantic==2.7.1
numpy==1.26.4
```

2c. .env file

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=dcia_poc_pass
OPENAI_API_KEY=sk-...          # or ANTHROPIC_API_KEY
LLM_PROVIDER=openai            # or anthropic
```

2d. neo4j_service.py

```python
```

```python
from neo4j import GraphDatabase
import os

driver = GraphDatabase.driver(
    os.getenv("NEO4J_URI"),
    auth=(os.getenv("NEO4J_USER"), os.getenv("NEO4J_PASSWORD"))
)

def get_cascade_impacts(system_id: str, max_hops: int = 3) -> list[dict]:
    with driver.session() as session:
        result = session.run("""
            MATCH path = (s:ShipSystem {id: $id})-[:CASCADES_TO*1..$hops]->(affected
            WITH affected, length(path) AS hops,
                 [r IN relationships(path) | r.reason] AS reasons,
                 [r IN relationships(path) | r.severity] AS severities
            RETURN affected.id AS id,
                   affected.name AS name,
                   affected.discipline AS discipline,
                   hops,
                   reasons,
                   severities[-1] AS severity
            ORDER BY hops
        """, id=system_id, hops=max_hops)
        return [dict(r) for r in result]

def get_class_rules_for_systems(system_ids: list[str]) -> list[dict]:
    with driver.session() as session:
        result = session.run("""
            MATCH (s:ShipSystem)-[:GOVERNED_BY]->(cr:ClassRule)
            WHERE s.id IN $ids
            RETURN s.name AS system, cr.society AS society,
                   cr.clause AS clause, cr.description AS description
        """, ids=system_ids)
        return [dict(r) for r in result]

def get_similar_historical_changes(keywords: list[str]) -> list[dict]:
    with driver.session() as session:
        # Simple keyword match for PoC; replace with vector similarity in production
        result = session.run("""
            MATCH (hc:DesignChange)
            WHERE any(kw IN $keywords WHERE toLower(hc.description) CONTAINS toLower
            OPTIONAL MATCH (hc)-[:GENERATED]->(cr:CostRecord)
            RETURN hc.id AS id, hc.description AS description,
```

```
                hc.cost_actual_usd AS cost_actual_usd,
                hc.schedule_delay_days AS delay_days,
                hc.summary AS summary,
                collect(cr.category + ':' + toString(cr.amount_usd)) AS cost_brea
            LIMIT 3
        """, keywords=keywords)
        return [dict(r) for r in result]
```

2e. llm_service.py

```python
```

```python
import os, json
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

SYSTEM_PROMPT = """You are DCIA, an expert AI assistant for a naval shipbuilding com
Your role is to analyse design change requests and provide structured impact assessm

When given a design change description, extract:
- system: the primary ship system being changed (must map to one of: propulsion, str
  electrical, weapons, outfitting, shafting)
- system_id: the Neo4j node id (SYS-001=propulsion, SYS-002=structure, SYS-003=elect
  SYS-004=weapons, SYS-005=outfitting, SYS-006=shafting, SYS-007=fuel)
- change_type: category of change (power_upgrade, weight_change, replacement, additi
- magnitude: percentage or absolute change where stated
- keywords: 2-4 keywords for finding similar historical changes

Respond ONLY with valid JSON. No preamble."""

def extract_intent(user_message: str) -> dict:
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_message}
        ],
        response_format={"type": "json_object"}
    )
    return json.loads(response.choices[0].message.content)

SYNTHESIS_PROMPT = """You are DCIA, a design change impact analyst for a naval shipb
Synthesise the provided impact data into a clear, professional assessment.
Always cite: affected systems, class rule references, cost estimate range, schedule
Be specific. Flag high-severity items prominently. Keep response under 400 words."""

def synthesise_report(user_query: str, impact_data: dict) -> str:
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content": SYNTHESIS_PROMPT},
            {"role": "user", "content": f"""
User query: {user_query}
```

```
Impact graph results: {json.dumps(impact_data['impacts'], indent=2)}

Class rule flags: {json.dumps(impact_data['class_rules'], indent=2)}

Similar historical changes: {json.dumps(impact_data['historical'], indent=2)}

Cost estimate: {json.dumps(impact_data['cost_estimate'], indent=2)}

Generate a structured impact assessment report.
"""}
        ]
    )
    return response.choices[0].message.content
```

2f. cost_model.py (parametric model for PoC)

```python
```

```python
# Simplified parametric model for PoC
# In production: replace with ML model trained on MDL historical data

DISCIPLINE_COST_FACTORS = {
    "propulsion":  {"base_usd": 2500000, "per_percent_usd": 85000},
    "structure":   {"base_usd": 180000,  "per_percent_usd": 12000},
    "electrical":  {"base_usd": 95000,   "per_percent_usd": 8000},
    "weapons":     {"base_usd": 450000,  "per_percent_usd": 35000},
    "outfitting":  {"base_usd": 65000,   "per_percent_usd": 4500},
    "shafting":    {"base_usd": 120000,  "per_percent_usd": 9000},
    "fuel":        {"base_usd": 55000,   "per_percent_usd": 3500},
}

SEVERITY_MULTIPLIERS = {"HIGH": 1.0, "MEDIUM": 0.4, "LOW": 0.15}

def estimate_cost(primary_discipline: str, magnitude_pct: float,
                  affected_systems: list[dict]) -> dict:
    primary = DISCIPLINE_COST_FACTORS.get(primary_discipline, {"base_usd": 200000, "
    base = primary["base_usd"] + (magnitude_pct * primary["per_percent_usd"])

    cascade_cost = sum(
        DISCIPLINE_COST_FACTORS.get(s["discipline"], {"base_usd": 50000})["base_usd"
        * SEVERITY_MULTIPLIERS.get(s.get("severity", "LOW"), 0.15)
        for s in affected_systems
    )

    total_p50 = base + cascade_cost
    return {
        "primary_system_usd": round(base),
        "cascade_systems_usd": round(cascade_cost),
        "total_p10_usd": round(total_p50 * 0.75),
        "total_p50_usd": round(total_p50),
        "total_p90_usd": round(total_p50 * 1.35),
        "confidence": "MEDIUM",
        "note": "PoC parametric model — replace with MDL historical data regression
    }
```

2g. main.py — the core chat endpoint

```python
```

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from services.neo4j_service import (
    get_cascade_impacts, get_class_rules_for_systems,
    get_similar_historical_changes
)
from services.llm_service import extract_intent, synthesise_report
from services.cost_model import estimate_cost

app = FastAPI(title="DCIA PoC API")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                   allow_methods=["*"], allow_headers=["*"])

class ChatRequest(BaseModel):
    message: str

@app.post("/api/chat")
async def chat(req: ChatRequest):
    # Step 1: Extract intent
    intent = extract_intent(req.message)

    # Step 2: Graph traversal — get cascading impacts
    impacts = get_cascade_impacts(intent.get("system_id", "SYS-001"), max_hops=3)

    # Step 3: Class rules for all affected systems
    affected_ids = [i["id"] for i in impacts] + [intent.get("system_id", "SYS-001")]
    class_rules = get_class_rules_for_systems(list(set(affected_ids)))

    # Step 4: Historical analogues
    historical = get_similar_historical_changes(intent.get("keywords", []))

    # Step 5: Cost estimate
    magnitude = float(intent.get("magnitude", "10").replace("%", "").strip() or 10)
    cost = estimate_cost(intent.get("system", "propulsion"), magnitude, impacts)

    # Step 6: LLM synthesis
    all_data = {
        "impacts": impacts,
        "class_rules": class_rules,
        "historical": historical,
        "cost_estimate": cost
    }
```

```
    report = synthesise_report(req.message, all_data)

    return {
        "intent": intent,
        "impacts": impacts,
        "class_rules": class_rules,
        "historical": historical,
        "cost_estimate": cost,
        "report": report
    }

@app.get("/api/health")
async def health():
    return {"status": "ok"}
```

```bash
# Run the backend
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# API docs at http://localhost:8000/docs
```

Step 3 — React.js front end

3a. Scaffold the app

```bash
cd frontend
npx create-react-app dcia-ui --template typescript
cd dcia-ui
npm install axios recharts react-flow-renderer lucide-react tailwindcss
```

3b. Core chat + impact display component

```tsx
```

```
// src/App.tsx
import { useState } from "react";
import axios from "axios";
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer, Cell } from "rec

const API = "http://localhost:8000/api";

const SEVERITY_COLORS: Record<string, string> = {
  HIGH: "#E24B4A", MEDIUM: "#EF9F27", LOW: "#1D9E75"
};

type ImpactResult = {
  intent: any; impacts: any[]; class_rules: any[];
  historical: any[]; cost_estimate: any; report: string;
};

export default function App() {
  const [message, setMessage] = useState("");
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState<ImpactResult | null>(null);
  const [error, setError] = useState("");

  const submit = async () => {
    if (!message.trim()) return;
    setLoading(true); setError(""); setResult(null);
    try {
      const res = await axios.post(`${API}/chat`, { message });
      setResult(res.data);
    } catch (e: any) {
      setError("API error: " + (e.message || "unknown"));
    } finally { setLoading(false); }
  };

  const costData = result ? [
    { name: "P10 (optimistic)", value: result.cost_estimate.total_p10_usd },
    { name: "P50 (likely)",     value: result.cost_estimate.total_p50_usd },
    { name: "P90 (pessimistic)",value: result.cost_estimate.total_p90_usd },
  ] : [];

  return (
    <div style={{ maxWidth: 900, margin: "0 auto", padding: "2rem", fontFamily: "san
      <h1 style={{ fontSize: 22, fontWeight: 500 }}>DCIA — Design Change Intelligenc
      <p style={{ color: "#666", fontSize: 14 }}>PoC · MDL Shipbuilding · Source: PS
```

```
{/* Input */}
<div style={{ display: "flex", gap: 8, marginTop: 24 }}>
  <input
    value={message}
    onChange={e => setMessage(e.target.value)}
    onKeyDown={e => e.key === "Enter" && submit()}
    placeholder='Try: "Increase main engine power by 15%. What systems are aff
    style={{ flex: 1, padding: "10px 14px", borderRadius: 8,
             border: "1px solid #ddd", fontSize: 15 }}
  />
  <button onClick={submit} disabled={loading}
    style={{ padding: "10px 20px", borderRadius: 8, background: "#185FA5",
             color: "#fff", border: "none", cursor: "pointer", fontWeight: 500
    {loading ? "Analysing…" : "Analyse"}
  </button>
</div>

{/* Sample prompts */}
<div style={{ marginTop: 10, display: "flex", gap: 8, flexWrap: "wrap" }}>
  {[
    "Increase main engine power by 15%",
    "Replace shaft generator with larger 3MW unit",
    "Add new weapons mount on deck 2",
    "Extend fuel tank capacity by 20%"
  ].map(p => (
    <button key={p} onClick={() => setMessage(p)}
      style={{ padding: "4px 12px", borderRadius: 20, border: "1px solid #ccc"
               background: "#f8f8f8", cursor: "pointer", fontSize: 13 }}>
      {p}
    </button>
  ))}
</div>

{error && <p style={{ color: "#E24B4A", marginTop: 16 }}>{error}</p>}

{result && (
  <div style={{ marginTop: 32 }}>

    {/* AI report */}
    <section style={{ background: "#f0f4f8", borderRadius: 10, padding: 20, ma
      <h2 style={{ fontSize: 16, fontWeight: 500, marginTop: 0 }}>Impact asses
      <p style={{ lineHeight: 1.7, fontSize: 15, whiteSpace: "pre-wrap" }}>{re
      <p style={{ fontSize: 12, color: "#888", marginBottom: 0 }}>
```

```
        Source: Neo4j graph traversal + LLM synthesis · Confidence: {result.co
      </p>
    </section>

    {/* Affected systems */}
    <section style={{ marginBottom: 24 }}>
      <h2 style={{ fontSize: 16, fontWeight: 500 }}>Affected systems ({result.
      <table style={{ width: "100%", borderCollapse: "collapse", fontSize: 14
        <thead>
          <tr style={{ background: "#f0f0f0" }}>
            {["System", "Discipline", "Hops from origin", "Severity", "Reason"
              <th key={h} style={{ padding: "8px 12px", textAlign: "left", fon
            ))}
          </tr>
        </thead>
        <tbody>
          {result.impacts.map((imp, i) => (
            <tr key={i} style={{ borderBottom: "1px solid #eee" }}>
              <td style={{ padding: "8px 12px" }}>{imp.name}</td>
              <td style={{ padding: "8px 12px", textTransform: "capitalize" }}
              <td style={{ padding: "8px 12px" }}>{imp.hops}</td>
              <td style={{ padding: "8px 12px" }}>
                <span style={{ padding: "2px 10px", borderRadius: 12,
                               background: SEVERITY_COLORS[imp.severity] + "22
                               color: SEVERITY_COLORS[imp.severity],
                               fontWeight: 500, fontSize: 12 }}>
                  {imp.severity}
                </span>
              </td>
              <td style={{ padding: "8px 12px", color: "#555" }}>
                {imp.cascade_reasons?.[imp.cascade_reasons.length - 1] || "—"}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </section>

    {/* Cost estimate */}
    <section style={{ marginBottom: 24 }}>
      <h2 style={{ fontSize: 16, fontWeight: 500 }}>Cost estimate (USD)</h2>
      <ResponsiveContainer width="100%" height={180}>
        <BarChart data={costData} layout="vertical">
          <XAxis type="number" tickFormatter={v => `$${(v/1e6).toFixed(1)}M`}
```

```
        <YAxis type="category" dataKey="name" width={140} fontSize={12}/>
        <Tooltip formatter={(v: number) => `$${v.toLocaleString()}`}/>
        <Bar dataKey="value" radius={[0,4,4,0]}>
          {costData.map((_, i) => (
            <Cell key={i} fill={["#1D9E75","#185FA5","#E24B4A"][i]}/>
          ))}
        </Bar>
      </BarChart>
    </ResponsiveContainer>
    <p style={{ fontSize: 12, color: "#888" }}>{result.cost_estimate.note}</
  </section>

  {/* Class rules */}
  <section style={{ marginBottom: 24 }}>
    <h2 style={{ fontSize: 16, fontWeight: 500 }}>
      Class rule flags ({result.class_rules.length})
    </h2>
    {result.class_rules.map((cr, i) => (
      <div key={i} style={{ padding: "10px 14px", borderLeft: "3px solid #EF
                            background: "#fffbf0", borderRadius: 6, marginBo
        <strong style={{ fontSize: 13 }}>{cr.society} — {cr.clause}</strong>
        <p style={{ margin: "4px 0 0", fontSize: 13, color: "#555" }}>{cr.de
        <p style={{ margin: "4px 0 0", fontSize: 12, color: "#888" }}>
          Triggered by: {cr.system}
        </p>
      </div>
    ))}
  </section>

  {/* Historical analogues */}
  {result.historical.length > 0 && (
    <section>
      <h2 style={{ fontSize: 16, fontWeight: 500 }}>Historical analogues</h2
      {result.historical.map((h, i) => (
        <div key={i} style={{ padding: "10px 14px", border: "1px solid #e0e0
                              borderRadius: 8, marginBottom: 8 }}>
          <strong style={{ fontSize: 13 }}>{h.id}</strong> — {h.description}
          <p style={{ margin: "4px 0", fontSize: 13, color: "#555" }}>{h.sum
          <p style={{ margin: "4px 0 0", fontSize: 12, color: "#185FA5" }}>
            Actual cost: ${h.cost_actual_usd?.toLocaleString()} ·
            Schedule delay: {h.delay_days} days
          </p>
        </div>
      ))}
```

```
            </section>
          )}
        </div>
      )}
    </div>
  );
}
```

```bash
# Run the frontend
cd frontend/dcia-ui
npm start
# Opens at http://localhost:3000
```

Step 4 — PoC test scenarios

Run each of the following prompts through the UI and verify the expected outputs:

Scenario 1 — Propulsion power upgrade (PS-1 primary use case)

Input: "We need to increase main engine power by 15% for improved speed. What are the downstream impacts and estimated cost?" **Expected:** Impacts on SYS-002 (Structure), SYS-006 (Shafting), SYS-003 (Electrical), SYS-005 (Outfitting) · Class rules CR-LR-001, CR-LR-002, CR-DNV-001 flagged · Historical analogue HC-2022-014 returned · Cost P50 ~$3.5–4.2M

Scenario 2 — Generator replacement (PS-2 BOM/compliance)

Input: "Replace the shaft generator with a new 3MW unit. What class approvals are needed and will the BOM need updating?" **Expected:** Impacts on SYS-003 (Electrical), SYS-004 (Weapons via power bus) · Class rule CR-DNV-002 flagged · Historical analogue HC-2021-007 returned · BOM node BOM-002 referenced

Scenario 3 — Multi-hop cascade

Input: "Add structural stiffening at frames 40–48 to support a new weapons mount. What else is affected?" **Expected:** Structure → Outfitting cascade returned (SYS-005, LOW severity, pipe rerouting reason) · Weapons system compliance flagged

Scenario 4 — Fuel system change

Input: "Increase fuel tank capacity by 20% for extended patrol range." **Expected:** SYS-007 (Fuel)

as primary · SYS-002 (Structure, tank reinforcement) as cascade · Historical analogue HC-2023-031 returned · Lower cost estimate than propulsion scenarios

---

Step 5 — PoC acceptance criteria

#	Criterion	Pass condition
1	NL query returns structured response	Response within 10s, all 6 sections populated
2	Graph traversal accuracy	Propulsion change returns ⩾3 affected systems
3	Multi-hop propagation	At least one 2-hop cascade (e.g. Propulsion→Electrical→Weapons) visible
4	Class rule citation	⩾2 class rule nodes returned with society + clause reference
5	Historical analogue match	⩾1 historical change returned for engine/power keywords
6	Cost banding	P10/P50/P90 returned; P90/P10 ratio ⩾ 1.3
7	UI renders without error	All 4 UI sections display; no console errors
8	Response is cited	Report text references specific systems, rules, and historical IDs

---

Step 6 — Next steps after PoC

1. **Data ingestion:** Connect to actual MDL PLM (Teamcenter) via REST API to populate real component and BOM nodes — replace sample data
2. **Vector RAG:** Replace keyword historical search with pgvector embeddings of real change order PDFs
3. **Cost model training:** Export 3–5 years of SAP cost actuals for closed change orders; train regression model to replace parametric stub
4. **3D BOM sync:** Parser for STEP file exports from CAD; delta detection against Neo4j BOM nodes
5. **Auth + RBAC:** Add Azure AD / LDAP integration; role-gated endpoints for cost data
6. **Production Neo4j:** Migrate from Docker to Neo4j Aura Enterprise or on-prem Causal Cluster